Computer Communications 212 (2023) 390–406

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Verification of a multi-connectivity protocol for Tactile Internet applications

Delia Rico *, María-del-Mar Gallardo, Pedro Merino

ITIS Software, University of Malaga, Malaga, Spain

ARTICLE INFO

Keywords:
Multi-connectivity
Tactile internet
Transport protocol
Formal verification
Statistical Model Checking (SMC)

ABSTRACT

Tactile Internet refers to a network that enables real-time, high-reliability haptic communication and control between humans, machines, and objects over the Internet. Tactile Internet applications include the remote control of drones, cars or industrial tele-operation. In this context, the Multi-connection Tactile Internet Protocol (MTIP) is a novel multipath transport protocol designed to support the requirements of Tactile Internet applications in large, private mobile networks. The objective of this paper is to analyze and verify the correctness and performance of the MTIP protocol to ensure that the protocol functions correctly under different network scenarios and it is ready to meet the performance requirements of Tactile Internet applications. For that purpose, a two-step approach is employed to analyze and verify MTIP. In the first step, a formal model of MTIP is developed using timed automata and the UPPAAL tool is utilized to verify correctness properties represented as temporal formulas. In the second step, the performance of the protocol is analyzed using the statistical model checking features of UPPAAL (UPPAAL SMC) in scenarios that are difficult and expensive to reproduce in a real network. The results indicate that MTIP's model meets the specified temporal properties, and the performance evaluation showcases the potential and trade-off of using multiple paths to enhance the communication. Based on the analysis and verification results, the paper emphasizes the readiness of MTIP for real-world deployment and highlights its potential benefits for enhancing the performance of Tactile Internet applications.

1. Introduction

Tactile Internet (TI) refers to the network that allows humans to interact remotely with other humans or with machines in real-time transmission of haptic information (touch, actuation, motion, vibration, forces, surface texture, temperature, etc.). The new applications over the Tactile Internet include high-precision teleoperation; robots, cars, industrial equipment, drones; advanced interpersonal communications such as holograms; and immersive virtual reality. The mobile nature of the devices involved at the end points makes wireless communications, and particularly 5G mobile networks, a key part of the required infrastructure.

To support TI, current mobile networks and their end-to-end transport protocols should evolve to meet very tight requirements in terms of latency and reliability. For instance, as detailed in [1], immersive virtual reality requires end-to-end latency below 5 ms, and haptic feedback requires that this parameter be even closer to 1 ms. The required reliability for live haptic-enabled applications is usually above 99.999%.

Recent efforts in the design of 5G networks have addressed these requirements with the definition of support for Ultra-Reliable Low Latency Communications (URLLC). The URLLC features affect many components in the end-to-end path, including the radio segment, the transport and the 5G core. However, such improvements are not directly exposed to the applications on the end devices, where the *classic* TCP/IP stack is still the main interface with the 5G network.

Despite continuous improvements, the foundations and services offered by the popular TCP and UDP transport protocols were defined more than 40 years ago, when application domains like Tactile Internet were still very far from being conceived. For instance, TCP was designed to offer 100% reliability, causing long latencies depending on the quality of the path between the end-points (mainly addressing packet loss and reordering). Such quality issues could appear occasionally even over the best 5G network due to handovers, interference, network overload, etc. In such situations having an extra delay to achieve full reliability is not acceptable for many applications. On the opposite side, UDP does not add any specific quality mechanism to the underlying path, and thus does not provide any improvement of reliability.

Variants of the classic TCP/IP protocols that provide contributions that come closer to the Tactile Internet requirements are the Real-time Transport Protocol (RTP) and extensions like the Multipath RTP

* Corresponding author.
E-mail addresses: deliarico@uma.es (D. Rico), mdgallardo@uma.es (M. Gallardo), pmerino@uma.es (P. Merino).

https://doi.org/10.1016/j.comcom.2023.10.013
Received 22 June 2023; Received in revised form 13 September 2023; Accepted 18 October 2023
Available online 26 October 2023

(MPRTP) [2]. However, these protocols are focused on providing service for real-time multimedia with typically large amounts of data, instead of controlling limited-size packets with very strict constrained deadlines as in TI remote operations and sensing.

To address this gap between the features offered by mobile networks like 5G and the requirements of Tactile Internet applications, we presented in [3,4] the Multi-connection Tactile Internet Protocol (MTIP). MTIP is a multi-connectivity transport protocol that works under the following realistic and novel scenario to support TI applications over mobile networks.

The first assumption is *partial reliability* in TI. In general, TI applications require a set of packets to arrive on time, allowing some packet loss at the application layer if the packets miss their deadline. Hence, the transport protocol should be time sensitive aware but could also allow some packet loss. The second assumption is *multi-connectivity*. We expect new devices connected to mobile networks to be multi-homed and to connect to several mobile networks at the same time. Such multi-connectivity could include variants, such as several technologies (4G and 5G), potentially with several mobile operators, and be available with standard multiple Subscriber Identity Modules (multi-SIM) or with virtual SIMs (eSIMs). The final assumption is *large, private networks*. Many TI use cases, such as remote control of industrial machinery or manufacturing, are typically set in private network environments. Such networks provide specific mechanisms like security methods (e.g., GTP tunnels [5]) or synchronization (e.g., Precision Time Protocol (PTP) [6]), among others, that are available for the protocols and applications running on the network.

In this scenario, the Multi-connection Tactile Internet Protocol (MTIP) [3] is a novel transport protocol offers programmable quality to applications that need a dynamic compromise between latency, reliability, throughput and jitter, among other parameters. MTIP combines the management of multiple *sublinks* with context awareness to provide controller-to-device communication of sequenced time-sensitive packets with high (but partial) reliability and low latency. A sublink is an end-to-end connection between interfaces of multi-homed endpoints. The use of multiple interfaces allows sending several copies of the messages using the selection of the best subset of sublinks at the time of sending. Due to the importance of latency in TI applications, MTIP is deadline-aware and uses timestamps to discard expired data that is no longer useful according to application preferences. By using these timestamps, as well as sequence numbers, MTIP is able to control data exchanges and duplicate packets without flow control or congestion control, which may add undesired latency.

One challenge at this stage is to guarantee the correctness and performance of the algorithms to send and receive packets in MTIP at an early stage. Following the huge amount of work done in formal modeling and verification of communication protocols [7,8], we address the automated analysis of correctness and statistical performance. In that direction, we already did initial modeling and verification work of MTIP with the tool SPIN in [3]. SPIN is very efficient to verify the absence of deadlocks and non-progress loops, as well as complex properties described with Linear Temporal Logic (LTL). We produced models good enough to provide evidence of correctness; however, the resulting models had some shortcuts due to the lack of mechanisms to model time with PROMELA (SPIN's input language). In addition, we need to go a step further and conduct a performance evaluation based on the formal model, which enable us to have a more diverse and cost-effective evaluation compared to a real implementation. These aspects are discussed in Section 6. These are the main motivations behind this paper, where we present progress in the analysis and verification of MTIP using the tool UPPAAL. UPPAAL supports modeling of time and can be used to check correctness properties like deadlock and those described with TCTL (Timed Computation Tree Logic). Moreover, the tool also supports stochastic timed automata to analyze performance-related properties.

We anticipate that the UPPAAL model requires decisions to make it verifiable with standard computing resources to avoid the state explosion problem. For instance, we need to reduce the number of distinct data to be sent over MTIP to a minimum following the ideas of G. Holzmann [9] and P. Wolper [10]. We also need to limit the number of sublinks and the range of sequence numbers and timestamps preserving correctness. These abstraction decisions are discussed in Section 5.

In summary, the contributions of the paper with respect to [3] are the following:

1. **New property-oriented definition of MTIP:** We designed the MTIP protocol as a new transport protocol to exploit multi-connectivity over the Internet considering partial reliability of time-sensitive traffic as the main driver. In this paper, we describe the protocol functionalities by means of the desirable correctness and performance properties it should meet.
2. **New formal model:** We build a new formal model of MTIP in order to use the correctness and performance analysis features of UPPAAL. Compared with our previous SPIN model, we now make use of timed automata to reflect the use of time in the protocol. A major contribution of this paper is how to model sequence numbers, timestamps and multiple sublinks to be able to evaluate the properties while keeping the size of the model verifiable.
3. **Automated verification of correctness:** We use UPPAAL support to check basic correctness, such as the absence of deadlocks, and specific correctness properties of MTIP described with the temporal logic TCTL.
4. **Performance evaluation:** We use the UPPAAL Statistical Model Checking (SMC) feature to provide estimations of the statistical performance of the protocol with different sublink configurations and probability of losing or delaying packets.

The rest of the paper is organized as follows. Section 2 presents related work on Tactile Internet applications and protocols, and formal verification. Section 3 describes some relevant background information on the formal modeling and verification tool UPPAAL. Section 4 provides an overview of MTIP's requirements, general architecture, and data management procedures. Then, Section 5 presents the modeling abstraction decisions taken for the model, while Section 6 completes the description of the model, presents the formal verification and the analysis of performance properties carried out in UPPAAL. Finally, Section 7 contains conclusions and future work.

2. Related work

2.1. Tactile internet applications and protocols

Tactile Internet (TI) applications are described by the IEEE Tactile Internet Standards Working Group [1] as applications for the remote control and manipulation of real or virtual devices such as drones, industrial robots, vehicles, etc. Ultimately, TI applications define critical use cases with strict requirements in terms of several Key Performance Indicators (KPIs) [11]. Typical TI demands range from requiring less than 10 ms latency to demanding more than 99.999% reliability, usually simultaneously. However, these are not the only objective KPIs, since TI use cases can have strict requirements in terms of specific indicators, depending on the application. For instance, the remote operation of industrial machinery usually requires low levels of jitter, and remote driving needs bigger service area dimensions. Some ranges of specific requirements for the remote control of TI applications are shown in Table 1. They are defined not only by the IEEE but also by additional international standardization organizations, such as the 3GPP, through 3GPP Cyber-Physical Systems (CPS) [12] and service requirements for the 5G systems [13,14].

Regarding network protocols, and specifically transport protocols, real-time protocols like RTP have been typically used for remote control

Table 1
Tactile Internet application KPIs and considerations.

TI application	Latency	Reliability	Other considerations
Industrial operation	1–10 ms (highly dynamic) 1–100 ms (dynamic)	99.99%–99.9999%	Packet size: Small (40–250 bytes) Data rate: Low (1–10 Mbps) Service area: Small (Few hundred meters) Other key aspects: Low Jitter (1–100 us)
Remote driving	0.5–2 ms (life-critical) 5–10 ms (dynamic)	99.9%–99.999%	Packet size: Medium (50–1000 bytes) Data rate: Medium (1–25 Mbps) Service area: High (Some Kms) Other key aspects: Wireless links
Drone control	0.5–2 ms (kinesthetic) 10 ms (tactile)	99.9%–99.999%	Packet size: Small (40–250 bytes) Data rate: Low (1–10 Mbps) Service area: High (Some Kms) Other key aspects: Wireless links
Remote surgery	5–10 ms	99.9999%	Packet size: Medium (50–1000 bytes) Data rate: Low (~ 1 Mbps) Service area: High (Some Kms) Other key aspects: High availability (99.999%)

applications [15,16]. However, it is arguable that the characteristics of remote control data for TI applications differ from the peculiarities of multimedia real-time streams, since the remote control packets are usually small and are sent at lower data rates that may allow avoiding flow control and retransmissions in order to reduce latency. That is why some extensions of transport protocols like Smoothed SCTP [17], and novel protocols like IRTP [18] or ETP [19], have been developed for the remote control operation use case, among others [20]. However, these protocols do not take advantage of one highly relevant trend of novel networks, namely the availability of multiple paths [21,22]. Protocols like Multipath TCP (MPTCP) [23] or Multipath QUIC (MPQUIC) [24] have addressed the fact that there are more multi-homed networks and devices that can highly benefit from protocols that support this operation. Although these protocols to exploit multiple paths are not within the scope of TI teleoperation, they highlight the importance and potential of the use of multiple paths to enhance communications. A multipath extension that may be more relevant to the scope of TI is Multipath RTP (MPRTP) [2]. Nevertheless, like RTP, current work in respect to MPRTP is still focused on high bit-rate streaming multimedia content between endpoints.

Compared to these protocols, the Multi-connection Tactile Internet Protocol takes advantage of the availability of multiple paths, providing mechanisms based on the duplication of packets. At the same time, it is aware of TI specific requirements, using application preferences and network status measurements in order to adapt to the deadlines of the different applications in a lightweight manner. In Section 4, we provide more details about MTIP.

2.2. Modeling and analysis of communication protocols

The formal modeling and verification of protocols is a powerful tool. It allows verifying properties and performing in-depth analysis of the correct operation of protocols. Its potential is shown by multiple studies in the literature that have used model-based automated analysis to validate correctness properties of different well known protocols and algorithms, from the more traditional, such as BGP [25] and CSMA [26], to novel ones, such as the 5G-AKA Protocol [27].

SPIN, the tool used in our previous work in [3], is a model checker for the logical verification of concurrent software, initially defined by G. Holzmann [9]. Since its origin, it has been used in a number of academic and industrial applications and protocols [28–31]. However, even if SPIN is a powerful tool to check the correctness of a model, SPIN does not support any performance evaluation. UPPAAL is also a model checking tool to analyze real-time systems modeled as networks of timed automata [32]. Like SPIN, UPPAAL has been used to analyze both traditional and novel communication protocols. Examples include the work of Saini et al. [33] evaluating the Stream Control Transmission Protocol; Rashid et al. [34], verifying the ZigBee protocol routing capabilities; and Wu et al. [35], analyzing the design of a protocol for drone flight control. It is also used for real-time performance analysis, such as the case of Wang et al. [36] evaluating the real-time performance of Time-Sensitive Networking Traffic Shapers.

Many communication protocols display some probabilistic behavior and are subject to a stochastic nature, so some additional statistical performance evaluation should be done. UPPAAL SMC (Statistical Model Checking) combines model checking with testing techniques for the purpose of also analyzing the performance of the models [37]. Examples of the use of UPPAAL SMC are the work of Houimli et al. [38] with a performance analysis of an Internet of Things protocol; Höfner et al. [39], which utilizes statistical model checking to evaluate wireless mesh routing protocols; and Naeem et al. [40], which presents a performance analysis of cyber–physical systems and their energy-related behavior.

In this work, we use the model checker UPPAAL and our previous knowledge of modeling MTIP with the SPIN tool [3] to create a new model of MTIP that allows us to verify not only correctness but also performance aspects of the protocol.

3. The UPPAAL tool for modeling and analysis

In this section, we summarize the main features of UPPAAL to model and analyze the MTIP protocol.

UPPAAL is a toolbox for the verification of real-time systems jointly developed by Uppsala University and Aalborg University [32,41]. UPPAALis a model checker able to *automatically* analyze complex properties on concurrent software such as communication protocols.

As is usual in model checkers, UPPAAL has two inputs: an abstract model of the system to be analyzed, described as a network of timed automata, and the desirable properties, specified using a subset of the timed computational tree logic TCTL. UPPAAL can *simulate* the model, which is useful to observe if its behavior corresponds to the expected one. In addition, the tool can also *verify* the model against the specified properties, producing a counter-example when a malfunction is found. UPPAAL's verification engine is based on the implicit storage of all state space produced by the model of the system under analysis.

Model checkers, in general, are specially good at finding hidden errors in concurrent systems. This is why the input modeling languages of these tools, as the networks of timed automata managed by UPPAAL, are designed to support concurrency and non-determinism.

Although UPPAAL suffers from the well-known state explosion problem, which occurs when the model analyzed produces too many states, the tool is, in fact, a very efficient model checker able to handle complex systems like the MTIP protocol presented in this paper.

An UPPAAL model is a network of the so-called *timed automata* which execute in parallel, communicate and synchronize each other via global

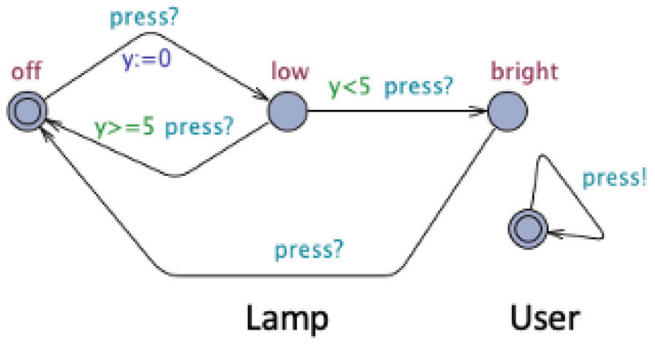


Fig. 1. The simple Lamp example in UPPAAL.

shared variables and synchronous channels Channels may also be declared as *broadcast* to distribute a message to more than one automaton.

In this Section, we briefly describe the main elements of UPPAAL needed to understand the paper. To clarify the presentation, we use a simple model of a lamp in UPPAAL extracted from [41], shown in Fig. 1, and the `Sender` process of MTIP protocol shown in Fig. 11 in Section 6.

The model of the `Lamp` in Fig. 1 contains a network of two timed automata: the `Lamp` on the left, and the lamp `User` on the right. In this system, the `User` *presses* the `Lamp` button to turn it `on` or `off`, depending on the `Lamp` state. The more interesting case occurs when the `Lamp` is `on` at a `low` intensity and the `User` presses the button. Then, if the `Lamp` has been `on` for less than 5 time units, the `Lamp` increases its intensity to `bright`. Otherwise, it turns `off`. Variable y is a clock used to record time, as explained later. Each timed automaton is composed of *locations* (the automaton states) and transitions. Locations are written as circles in the automata (for instance, the nodes labeled with `off`, `low` and `bright` in Fig. 1 are the locations of the `Lamp` automaton). Initial locations are represented with a double circle. Each automaton must only have one initial location. For instance, location `off` is the initial location of automaton `Lamp`. Transitions between locations are represented by means of arrows which can be decorated with different elements as described below.

Timed automata can also have global and local variables to store the automata network configuration. Besides the discrete variables common in computational systems, the *timed automata* of UPPAAL may also contain the so-called *clock* variables, which are dense and real-valued variables whose values evolve linearly with time. For instance, variable `y` in the `Lamp` automaton of Fig. 1 is a clock used to record the time passed at location `low`. Similarly, variable `clockTime` of automaton `Sender` of Fig. 11 is a global clock used to register the global time. Locations can have *invariants* which are Boolean expressions that have to be `true` while the automaton is at the corresponding location. When the invariant of a location is `true` (that is, no condition is imposed to stay in the location) then no Boolean expression is written next to the location as occurs in the three locations of the automaton `Lamp`. Otherwise, the invariant expression is written next to the location. For instance, location `ReadyToSend` of Fig. 11 has `clockTime<=TIMELIMIT && clockTime<=sndTime + 1` as an invariant.

As commented above, automata transitions can contain different elements. A transition may have a *guard* that is a Boolean condition. An automaton can only take a transition if its guard evaluates to `true`. For instance, in the `Lamp`, guards `y<5` and `y>=5` in transitions from `low` to `bright`, and `low` to `off` establish the `y` values allowed to take each transition. Similarly, in Fig. 11, the transition from `ToSublinks` to `ReadyToSend` can only be executed if the guard `sblIterator==c_sblNum` is `true`.

A transition may also have a *synchronized* condition which is expressed via an operation on a synchronized channel such as channel `press` in the `Lamp` example. Operations on channels are written using the CSP notation (`press!` and `press?` are, respectively, the write and read operations in the example). Channel operations constitute the natural mechanism for synchronizing timed automata in UPPAAL. A transition with a synchronized condition can only be fired if the synchronization is possible at that moment. In the `Lamp` example, this synchronization is always available, since the `User` is continuously trying to press the button via operation `press!`. However, in the `Sender` in Fig. 11, the transition from location `WaitingForReset` to `ReadyToSend` can only take place if the synchronization via channel `Reset` is available.

Finally, transitions can also be used to *update* some variables. For instance, in the `Lamp`, the transition from `off` to `low` reset clock `y` to 0, is used to measure the exact time the automaton spends in state `low`. Transitions can update any variable. For example, in the `Sender` Fig. 11, the transition from `Sending` to `SelectingRed` updates variable `sndData` to `RED` and increases variable `wolper` by one. Finally, it is worth noting that a transition cannot be executed if the invariant of the remote location is `false`.

Intuitively, the execution of a network of timed automata proceeds as follows. All network automata start at their initial location. The network can evolve in two ways: (1) by means of a *continuous network transition*, where the time of all clocks increases simultaneously or (2) by means of a *discrete network transition*, when one (or several) automaton transits to a remote location. It is important to note that discrete transitions are *instantaneous*, meaning that their execution involves no time passing. Observe that when a transition contains a synchronization via a channel, it has to be executed simultaneously by all the automata involved in the synchronization. The network blocks when it is not possible to execute either a continuous or a discrete transition.

If `(loc,y=n)` denotes the state of automaton `Lamp` when it is at location `loc` and the value of `y` is `n`, the following two sequences of states show possible executions of the network of the `Lamp` example. In the sequence, $\xrightarrow{n}$ represents a continuous transition of duration n, while $\xrightarrow{\texttt{press}}_d$ is the discrete transition produced by synchronizing the `Lamp` and `User` transitions via channel `press`:

1. `(off,0)` $\xrightarrow{2}$ `(off,2)` $\xrightarrow{\texttt{press}}_d$ `(low,0)` $\xrightarrow{2.5}$ `(low,2.5)` $\xrightarrow{\texttt{press}}_d$ `(bright,2.5)` $\xrightarrow{1.5}$ `(bright,4)` $\xrightarrow{\texttt{press}}_d$ `(off,4)` ...
2. `(off,0)` $\xrightarrow{1.5}$ `(off,1.5)` $\xrightarrow{\texttt{press}}_d$ `(low,0)` $\xrightarrow{5}$ `(low,5)` $\xrightarrow{\texttt{press}}_d$ `(off,5)` ...

Note that in a timed automata network, there are several sources of non-determinism that may produce a non-numerable set of different behaviors. For instance, an automaton may take a transition at any moment during the time interval in which both the invariant and the transition guard are `true` and the transition synchronization, if it exists, is available. In addition, if several discrete transitions are possible, the system may evolve selecting any of them.

UPPAAL provides several mechanisms to control non-determinism. One of them is to declare locations as *committed*, which is denoted placing a C letter inside the location. When some automaton of the network is at a committed location, no continuous transition can be carried out, since the transitions from committed locations are executed with priority. Committed locations allow executing a sequence of transitions atomically. For instance, in the `Sender` automaton of Fig. 11, the sequence of discrete transitions from `Sending` to `SelectingRed`, then to `WolperSelected`, to `ToSubLinks`, etc., are atomically executed since all these locations are committed.

3.1. Temporal logic

UPPAAL can analyze properties written in a well defined subset of the timed computational tree logic TCTL which is a timed version of temporal logic CTL. Logic CTL is aimed at describing the expected behavior

$$
\begin{aligned}
\sigma &::= p \mid \texttt{false} \mid !\sigma \mid \sigma \| \sigma \\
f_{ctl} &::= \sigma \mid !f_{ctl} \mid f_{ctl} \| f_{ctl} \\
&\mid \texttt{E<>}f_{ctl} \mid \texttt{E[]}f_{ctl} \mid \texttt{E}f_{ctl}\texttt{U}f_{ctl} \\
&\mid \texttt{A<>}f_{ctl} \mid \texttt{A[]}f_{ctl} \mid \texttt{A}f_{ctl}\texttt{U}f_{ctl} \\
f_{uppaal} &::= \sigma \mid \texttt{E<>}\sigma \mid \texttt{E[]}\sigma \mid \texttt{A<>}\sigma \mid \texttt{A[]}\sigma \\
&\mid \texttt{A[]}(\sigma \texttt{ -> A<>}\sigma)
\end{aligned}
$$

Fig. 2. CTL syntax.

of a system on the execution tree it determines from its initial state. To do this, the semantics of CTL formulae rely on the formalization of systems using transition systems as $S = (S, s_0, \rightarrow)$ where S is the set of system states, $s_0 \in S$ is the initial state, and $\rightarrow \subseteq S \times S$ defines the transitions $s \rightarrow s'$ between states. Informally, from relation $\rightarrow$, we can inductively construct the execution tree $tree(s)$ for each state $s \in S$, as $tree(s) = \{s \rightarrow tree(s') | s \rightarrow s'\}$, i.e. $tree(s)$ is a tree with root s containing all possible executions determined by S from s. From $tree(s)$, we can define $paths(s)$ as the set of execution paths of the form $s \rightarrow s' \rightarrow \cdots$ that can be obtained traversing $tree(s)$ from its root s.

Fig. 2 shows part of the syntax of CTL. The rule for σ defines the syntax of the so-called state formulae which can be evaluated on single system states. In this rule, p ranges over a set of atomic propositions. The second rule defines the CTL formulae that can be either state formulae or make use of the universal and existential path quantifiers A and E. A means "for all paths" while E can be read as "for at least a path". In CTL, path quantifiers are always written next to the temporal operators "always" ([]) and "eventually" ($\langle\rangle$). Usually, the semantics of [] and $\langle\rangle$ are derived from the semantics of operator "until" U, but we have skipped it here to make this presentation easier, since it is not supported by UPPAAL. Operators [] and $\langle\rangle$ evaluate on execution paths with the following intuitive meaning: $[]f$ holds on a path $\pi = s_i \rightarrow s_{i+1} \rightarrow \cdots \in paths(s)$ iff for all $j \geq i$ the sub-trace $\pi^j = s_j \rightarrow \cdots$ satisfies f. Similarly, $\langle\rangle f$ holds on a path $\pi \in paths(s)$ iff there exists $j \geq i$ such that the sub-trace $\pi^j = s_j \rightarrow \cdots$ satisfies f.

Taking this into consideration, the semantics of the CTL formulae can be summarized as:

- σ holds on $tree(s)$ iff state s satisfies σ.
- $\texttt{E}\langle\rangle f_{ctl}$ holds on $tree(s)$ iff there exists some execution in $paths(s)$ for which $\langle\rangle f_{ctl}$ holds.
- $\texttt{E}[]f_{ctl}$ holds on $tree(s)$ iff there exists some execution path in $paths(s)$ for which $[]f_{ctl}$ holds.
- $\texttt{A}\langle\rangle f_{ctl}$ holds on $tree(s)$ iff $\langle\rangle f_{ctl}$ holds for all execution paths of $paths(s)$.
- $\texttt{A}[]f_{ctl}$ holds on $tree(s)$ iff $[]f_{ctl}$ holds for all execution paths of $paths(s)$.

The last rule of Fig. 2 contains the subset of TCTL formulae supported by UPPAAL. Formula $\texttt{A[]}(\sigma_1\texttt{->A<>}\sigma_2)$ is written as σ_1- ->σ_2 in the tool.

The following TCTL formulae describe some properties of the Lamp example of Fig. 1

1. "The Lamp is initially off": `Lamp.off`
2. "The Lamp may be bright": `E<>Lamp.bright`
3. "If the Lamp is bright then it can be off, eventually": `A[] (Lamp.bright -> E<>Lamp.off)`
4. "It is always possible that the Lamp is on": `A[]E<> (Lamp.low || Lamp.bright)`

3.2. UPPAAL SMC

UPPAAL SMC is an extension of UPPAAL able to carry out Statistical Model Checking (SMC) on networks of timed automata. SMC allows combining model checking and testing approaches to analyze systems that display a stochastic nature. To do this, UPPAAL automata and TCTL properties can be enriched with statistical annotations before being simulated and analyzed by UPPAAL SMC. For instance, a rational number of the form $a : b$ decorating a location associates it with an exponential probability distribution describing the chance of the automaton to leave the location. Transitions can also be decorated with constant values, indicating the probability that the automaton chooses the transition to leave the location. Finally, properties accepted by UPPAAL may be annotated as $\texttt{Pr[<=N]A<>}\sigma$ to ask for the probability of the system to satisfy $\texttt{A<>}\sigma$ before the global time reaches value N.

4. The Multi-connection Tactile Internet Protocol

The Multi-connection Tactile Internet Protocol is a multipath transport protocol for the remote control of Tactile Internet applications. MTIP creates a link between a multi-homed Tactile Internet device and a remote controller. This link consists of multiple sublinks that represent independent end-to-end connections through different wireless communication networks such as 4G, 5G or 6G, as shown in Fig. 3.

Fig. 3. Link and sublink structure in MTIP.

MTIP is geared at Tactile Internet communications with specific restrictive demands, where the traffic pattern usually includes short packets with latency and reliability constraints (see previous Table 1). In order to meet the strict requirements, one of MTIP's main characteristics is the use of multi-connectivity. By sending redundant packets over multiple sublinks, MTIP exploits the benefits of the different available paths and increases reliability without using other reliability mechanisms like retransmissions that impact negatively on the latency. In fact, due to the importance of latency requirements, MTIP is deadline-aware and uses timestamps to discard expired data that is no longer useful according to application preferences. By just using these timestamps and sequence numbers, MTIP is able to control data exchanges and duplicate packets without flow control or congestion control, which would add unnecessary artificial waits for such small packets and data rates. Moreover, the MTIP header remains light, as shown in Fig. 4.

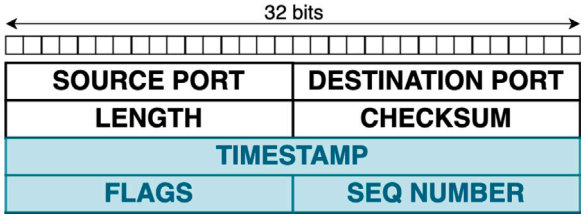


Fig. 4. The MTIP header.

The specific purpose of sequence numbers in MTIP is to avoid reorder and duplication when sending the data to the consumer application. Like in all protocols that use sequence numbers, their range is finite due to the actual limit of the protocol header, and some recycling is needed. We must ensure that recycled numbers cannot disturb the correct working of the protocol, avoiding a negative impact in low-latency applications. Hence, MTIP cannot benefit from the sliding window feature of other protocols like TCP to avoid a quick recycling of sequence numbers. We need the following premise when designing the cyclic sequence numbers in MTIP (the **recycling assumption**): *the valid maximum latency supported by a packet is always much lower than the time needed to recycle the sequence numbers*. So, even if two packets with the same sequence number are present at the consumer buffer, one of them will be discarded as a late packet. Note that this is a realistic

assumption as shown in Table 1, where the worst scenario for recycling would be the one with the maximum supported latency of 100 ms. With 16-bit sequence numbers, MTIP can support a maximum of 2^{16} control actions (messages) in that 100 ms slot to avoid two on-time packets with the same sequence number. This translates into sending a maximum of one action every 0.0015 ms (655360 packets/s). Holland et al. [42] show that this is a reasonable data rate for TI applications like tele-operation or automotive, which present the most extreme case with an action every 0.25 ms (4000 packets/s). In any case, longer sequence numbers could be used in specific applications at the cost of increasing the header.

MTIP is envisioned for large private networks, the typical environment of Tactile Internet use cases such as the remote control of industrial machinery or manufacturing. Large private networks, conjointly with novel cellular communications such as 5G, have the potential of offering an enhanced environment for such applications in terms of security, availability, optimization, etc. MTIP makes use of the specific mechanisms that private networks offer to function properly, such as security methods, like GTP tunnels in the fixed part of the network, and synchronization like the Precision Time Protocol (PTP). Moreover, such networks may also have specific methods to expose network information and enhance MTIP network measurements.

MTIP design properties can be separated into correctness and performance. Correctness properties describe how MTIP performs the basic exchange of data, while performance properties add constraints to this exchange of data in order to explore some beneficial trade-offs for the specific application. In the next subsections, we describe the four correctness and two performance properties considered for MTIP.

4.1. MTIP correctness properties

Apart from the restrictions of TI data, the use of multiple sublinks for a same connection entails potential issues that must be managed by the communication protocol. For instance, sending multiple copies of a same packet at the transport level should never result in receiving multiples copies on the receiving application. Moreover, multipath data transmissions may cause reordering due to several causes (q.v. [43]), such as consuming from different sublinks when packet loss occurs. Fig. 5 visually depicts these potential issues.

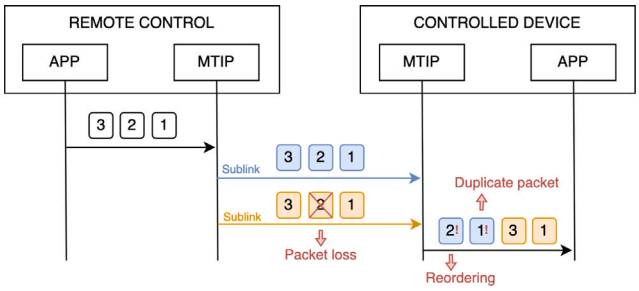


Fig. 5. Potential issues in multipath communications.

In order to verify the correct MTIP operation, we define four correctness properties that must be satisfied. For a better understanding, in this section data at the application level will be referred to as *messages*, while data at the transport level will be referred to as *packets*. The four correctness properties of MTIP are the following.

- **P1**: Application never receives *messages* that have met their deadline (*packets* that are considered late).
- **P2**: Application never receives duplicate *messages*.
- **P3**: Application never receives out-of-order *messages*.
- **P4**: Every *packet* that is not a duplicate or late is forwarded to the application.

To satisfy the correctness properties, MTIP uses the sequence numbers and timestamps in the following manner. First, when sending *messages*, MTIP dispatches duplicate *packets* over the different sublinks. To be able to identify those duplicates, each copy has the same sequence number as the original. This sequence number not only identifies duplicates, but also identifies reordering and losses, since sequence numbers are increased with each *message*. This operation is complemented with timestamps to control the life of *packets*. Each *packet* carries a timestamp marking the exact time when the *message* was generated. The timestamp will be used on the reception side to calculate the deadline of the *packet*, and additional network measurements to characterize the sublinks. The MTIP reception algorithm is depicted in Fig. 6.

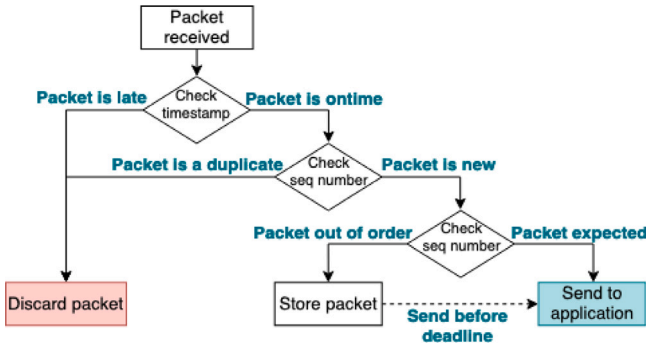


Fig. 6. The MTIP reception algorithm.

Upon reception of a *packet*, the reception algorithm computes the deadline of each *packet* using the timestamp and current time. Then, it uses a window that controls if the *packet* has met the deadline and is no longer relevant to the application layer. If the *packet* is late, it is discarded to satisfy **P1**. If the *packet* is not late, the next step is to control duplicates. MTIP stores the sequence numbers already received that are still relevant time-wise. Each new *packet* is compared with previously received sequence numbers, ensuring that if it is a duplicate, it is discarded and not sent to the application, satisfying **P2**. Finally, for **P3**, MTIP checks that there is no gap in the sequence numbers received. If there is a gap, MTIP stores the *packet* for a certain amount of time before sending it to the application. This time is calculated so that it never exceeds the deadline of the *packet* (refer to [43] for further information about multipath *packet* expiration). If the missing *packet* arrives within that time frame, both *packets* are sent to the application in order. If the missing *packet* does not arrive, it is considered lost and the stored *packet* is sent to the application. The combination of these operations satisfies **P4**. Fig. 7 shows a message sequence chart (MSC) that depicts a simplified data exchange in MTIP.

4.2. MTIP performance properties

MTIP performance properties define the performance of the protocol according to application preferences. While using all sublinks to send duplicate data would be the most beneficial in terms of maximizing some KPIs, like reliability, it could also incur in an excessive use of the resources that could affect both the network and the Tactile Internet devices. Hence, MTIP exhibits a trade-off between reliability, which refers to the correct reception of data by the consuming application, and resource utilization, which relates to the amount of duplicate packets employed during communication. This trade-off can be separated in two performance properties:

- **P5**: The use of more sublinks tends to result in more correct *messages* received.
- **P6**: The use of more sublinks tends to result in more duplicate *packets* received.

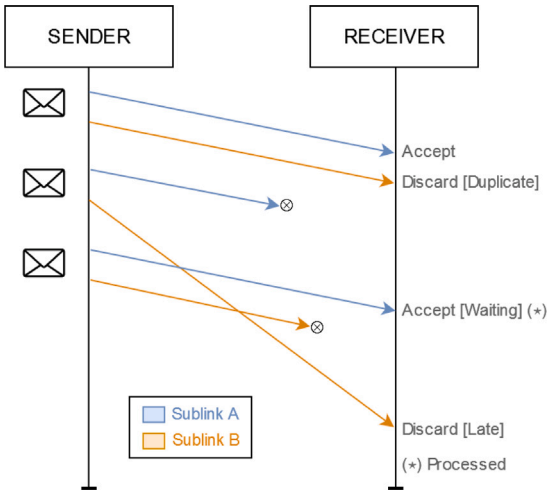


Fig. 7. Simplified MSC of MTIP.

In order to address this trade-off, MTIP exposes an Application Programming Interface (API) where the application can indicate its preferences. MTIP then uses this information, plus network measurements, to select which sublinks should be used to send data. Regarding application preferences, the application can define communication preferences such as the maximum latency that should be tolerated on the reception side. In terms of network awareness, MTIP performs continuous measurements on-the-go using the sequence numbers and timestamps. Thanks to these measurements and optional exhaustive ones, MTIP creates a database of KPI values, such as latency and reliability, for each sublink. MTIP presents a first algorithm to make the selection of the sublinks based on this information, which is presented in [4]; however, it also offers the application the possibility of making its own selection.

5. Modeling decisions

The main difficulty to use model checking is how to build an abstract model of the protocol that represents the relevant behaviors, but at the same time can be analyzed with the available resources (mainly system memory). As explained by G. Holzmman in [7,9], the protocol model for verification should only reflect the details to conclude that properties verified in the model are also satisfied in the real protocol. In addition, removing unnecessary details contributes to mitigate the state explosion problem. Practical rules for modeling include avoiding redundancy, sinks, sources, filters, or counters [44].

In our model of MTIP, such abstractions should additionally consider the size of relevant parameters such as application layer data, sequence numbers, timestamps and sublinks. This section provides our design decisions regarding these aspects.

5.1. Wolper test for application data

One major challenge when modeling MTIP is to decide how much application layer data should be sent over MTIP to preserve the correctness properties. This problem was already addressed by G. Holzmann [9] using previous results by P. Wolper [10]. Wolper developed the "data independence test" which reduces the amount of different data to be used to analyze the correctness of one algorithm. He demonstrated that verification results are valid for a given set of user data if the behavior of the underlying system managing these data does not depend on the specific set.

G. Holzmann adapted Wolper results to the case of end to end communication protocols in order to verify correctness properties. Holzmann stated that three distinct data items suffice for the typical correctness properties of flow control protocols (e.g., red data, blue data and white data), if they are sent in a sequence consisting of one red and one blue message inserted randomly in an infinite sequence of white messages. This number of minimum distinct data is independent of other factors, such as the sequence number range or the window size.

Fig. 8 represents part of a sender process in UPPAAL that implements the sequences of data defined by Holzmann to implement the Wolper test. It presents a loop that starts sending white messages (*SelectingWhite*) until a non-deterministic selection of UPPAAL increases the variable `wolper`. Then, it sends a red message (*SelectingRed*) and returns to sending white messages (*SelectingWhite*). At some point, the non-deterministic interleaving will increase the variable `wolper` again, and it will send a blue message (*SelectingBlue*). Finally, it concludes with an infinite sequence of white messages (*SelectingWhite*) and the Wolper test is considered completed. As for model checking, this code will be executed in an exhaustive manner (interleaving with the rest of the MTIP model), to ensure that all possible infinite sequences of white, red and blue messages are sent.

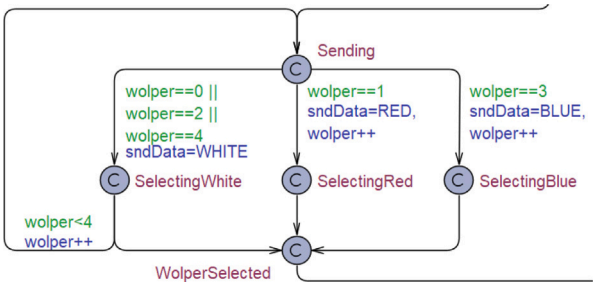


Fig. 8. Part of the `Sender` automaton: The Wolper test.

5.2. Modeling time

To model time in UPPAAL, we make use of clock variables that simulate time. The first abstraction decision in this context is to set a value that defines the DEADLINE of the packets. In MTIP design, this value is specified by the application (provided the **recycling assumption** is preserved). Higher numbers tend to provoke more packets arriving on time to the destination endpoint, while lower numbers favor more packets arriving late, due to the reduced time lapse it sets for the packets to arrive at the other endpoint. In our model, we have tested different DEADLINE values and assessed that the model always satisfies the correctness properties (**P1** to **P4**) and follows the same tendency for the performance properties (**P5** and **P6**). So, to simplify, we use a DEADLINE of three, the same number used in our previous model [3]. However, the results of the evaluations of the different DEADLINE values (1 to 4) wrt to the properties are accessible in [45].

As already stated, UPPAAL is able to handle real time by using clocks. However, UPPAAL uses time symbolically by means of constraints, or intervals [46]. This causes that clock values cannot be directly used in UPPAAL: they can be compared with integer expressions in guards, as in $y < 5$, for instance, but you cannot know the current value of a clock. For this reason, we need the support of extra variables such as `sndTime` to manage time in the UPPAAL implementation of the protocol. These variables work in conjunction with the clocks to effectively manage time but should not grow uncontrollably, so a `TIMELIMIT` must be set. This `TIMELIMIT` must be sufficient to allow the presence of packets that are still on time and packets that are late in the sublinks. In the next subsection, we focus on this specific value.

5.3. Sequence numbers

Following the **recycling assumption**, we need to find a valid abstraction to reduce the 2^{16} available sequence numbers in MTIP to a reasonable size for a formal verification. Our goal is to accurately model

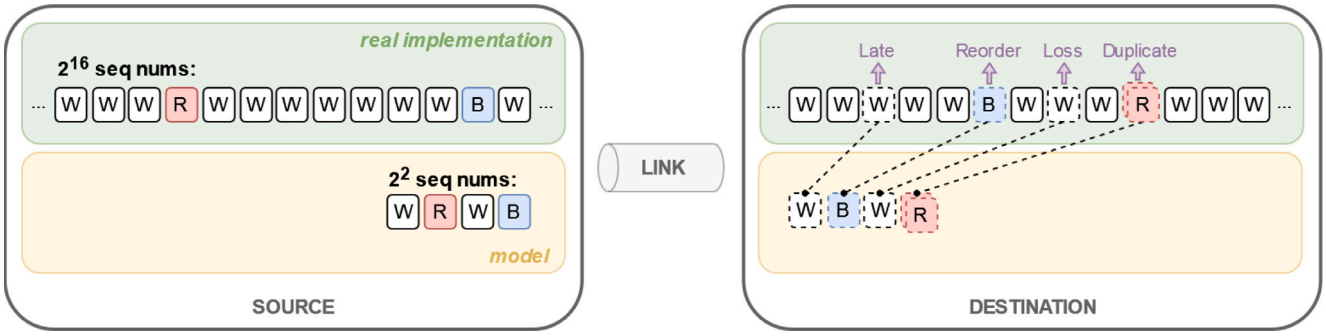


Fig. 9. Example of the abstraction of sequence numbers for the model.

and manage various scenarios that can occur in an MTIP link, including duplicate packets, packet reordering, packet loss, and late packets.

To achieve this, we take an incremental approach to determine a reasonable size for the sequence numbers. Firstly, we need at least one sequence number to detect duplicate packets. If the receiving side receives this sequence number twice, it can flag the packet as a duplicate. If we want to model packet reordering, we need at least two differentiated sequence numbers to enable the protocol to identify unexpected orders during processing. Additionally, to represent gaps in received data, we need an extra sequence number to abstract the previous situation plus some application data being lost. Finally, if we want to incorporate late application data, we need an additional sequence number. Therefore, to model the most extreme case where different application data is duplicated, reordered, lost, and late, we need a minimum of four sequence numbers to ensure that the protocol can recover from this scenario effectively. Fig. 9 displays an example that depicts this abstraction. It shows a sequence of white, red and blue data sent over a link (constituted by multiples sublinks) and received at a destination endpoint. In green, it shows duplicated, reordered, lost and late data using 16-bit sequence numbers, while in yellow it shows how this scenario will translate into 4-bit sequence numbers in order to be effectively abstracted.

Sequence numbers must have a limit and be incremental. In our model, the timestamps have the same behavior, since they need a `TIMELIMIT` and they are also incremental. In fact, we can increment each timestamp by one without limiting behaviors, since losses would simulate periods without sending data. That is why we just use timestamps (`sndTime` and `rcvTime`) to model both time and sequence numbers and set the `TIMELIMIT` to four. Moreover, this replicates the behavior of TI applications that normally send data in a constant and periodic manner [42] (e.g., 1 KHz). However, in a real implementation where timestamps are used to measure the specific end-to-end latency of each packet and packets might be sent with small variations in the period, it is necessary to use the complementary sequence numbers to ensure MTIP operation in a fail-safe manner.

5.4. Sublinks

The use of several sublinks can provoke packet reorders (unexpected sequence number received) and duplicates (same sequence number received from several sublinks) at the receiving side (see previous Fig. 5). It is clear that we need at least two sublinks to model reorders and duplicates. The question is whether more than two sublinks add any added value compared to two sublinks. Regarding the correctness properties, adding more sublinks would generate longer reorders and more duplicates, but the states generated with two sublinks are enough to check MTIP's ability to manage these situations. However, in terms of performance properties, even if two sublinks are enough, it is interesting to explore the impact of additional sublinks in the protocol's trade-off. Therefore, in our UPPAAL model, we use three sublinks.

6. Analysis and verification of MTIP with UPPAAL

In this section, we provide a description of the UPPAAL model for MTIP that has already been introduced in previous sections. Additionally, we evaluate the model by defining target properties in TCTL (Timed Computation Tree Logic). First, we assess correctness properties to validate MTIP's data exchange. Then, we verify performance properties using the UPPAAL SMC to analyze the use of a different number of sublinks under various network scenarios. Compared to the evaluation of the implementation of MTIP performed in [4], which focused on assessing the adaptability of MTIP for selecting the appropriate sublinks, this evaluation examines the effects of using a variable number of sublinks across a wider range of network scenarios. Performing this validation on statistical model checking allows extracting these insights in a more cost-effective manner than conducting experiments on a physical testbed. Moreover, it enables future adjustments to be made to the protocol algorithm in order to select a more appropriate number of sublinks based on the specific scenario.

6.1. The UPPAAL model

The UPPAAL model developed for MTIP is represented in Fig. 10. It presents an automaton `Sender` that uses `Sublinks` automata to communicate with an automaton `Receiver`. At the application level, it models a `Source` that represents a remote control and a `Destination` that represents a controlled device. Following recommendations for practical modeling [9], these applications are directly implemented in the transport `Sender` and `Receiver` automata to avoid state explosion. We now discuss some additional details of the model.

The `Sender` automaton is shown in Fig. 11. It is constituted by a main loop (bottom part) that continuously sends data through the different sublinks (indicated by the configurable value of `c_sblNum`). This data follows the Wolper test, which is directly implemented in the `Sender` automaton, and includes a timestamp `sndTime` in each packet. In the top part of the automaton, we can see the functionality to reset the value of `sndTime` when the `TIMELIMIT` is reached in all parts of the model.

The `Sublink` automaton, presented in Fig. 12, is responsible for simulating channel operation. It has two main locations, `Empty` and `notEmpty`. The `Sublink` initiates synced with all the other automata and stays at the location `Empty`. When the `Sender` wants to send some data to the other endpoint, it uses the `Sublink` automaton, which receives the data and stores it with its timestamp in a circular buffer. Then, the `Sublink` automaton progresses to the `notEmpty` location, where it can keep receiving data until it is full or has the opportunity to send the packet (data and timestamp) to the `receiver` automaton in a non-deterministic manner. The time it takes the automaton to leave this

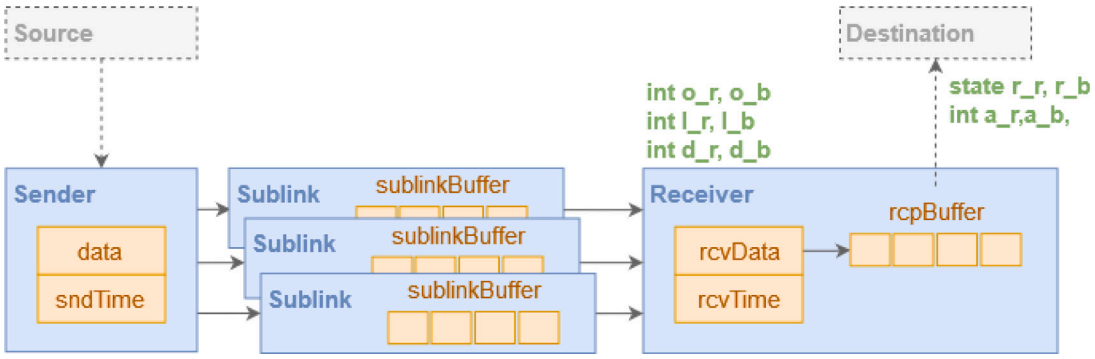


Fig. 10. Simplified graphical view of the UPPAAL model.

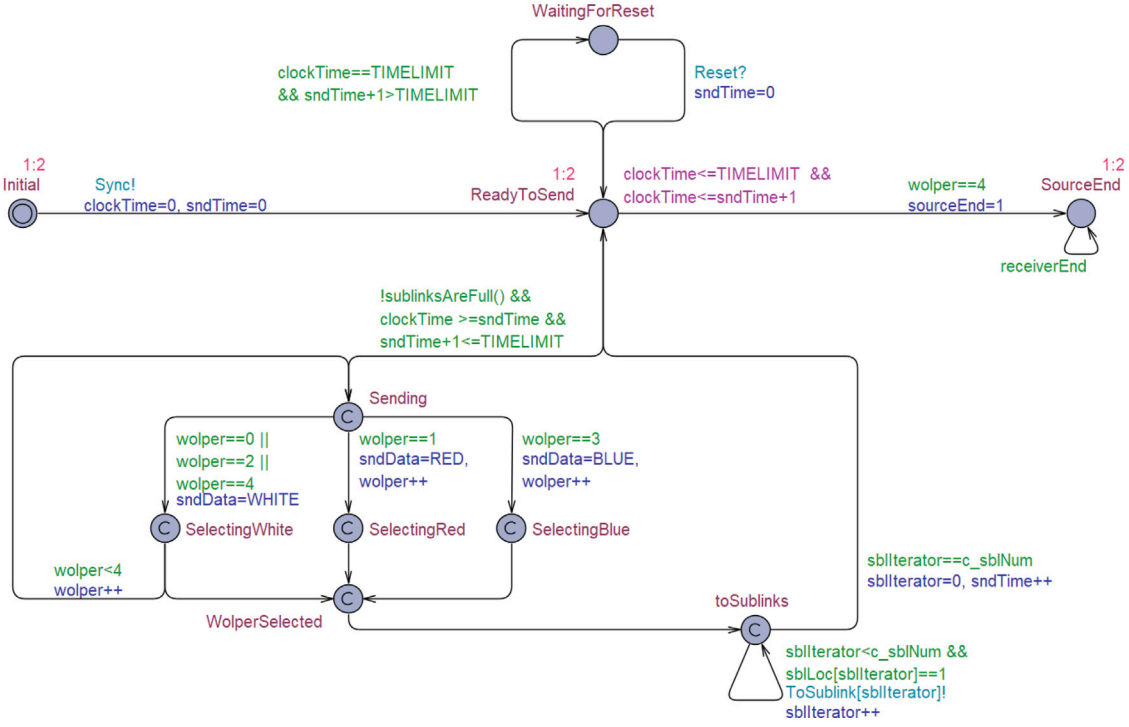


Fig. 11. `Sender` automaton.

location is configured by the exponential rate `c_lateTendency`, a variable that indicates the probability that the automaton will leave the location. The data can be successfully sent to the `Receiver` automaton, or the packet can be lost during the sending process. The selection of how data are sent is also non-deterministic at first; however, a variable `c_lossTendency` has been added in order to control the losses in terms of probability and study the impact of the use of multiple sublinks. Note that variables such as `dstLoc` or `sblLoc` are necessary since UPPAAL SMC only supports broadcast channels for synchronization, as described in [47]. The `Sublink` automaton also has an operation that forces discarding every packet that is left in the queues when the clock needs to be reset by the `Receiver` endpoint, since those packets will have already reached the deadline.

The `Receiver` automaton (Fig. 13) has the following tasks. First, it initiates the synchronization with all the automata (left part). Then, it waits for packets to be received from any sublink. When packets are received (bottom part), it checks if the packet is a duplicate, on time, late or lost and it only saves it in the buffer if it is an accepted packet according to the MTIP receiving algorithm explained in Section 4.1. Note that there are additional variables (`a_r, a_b, o_r, o_b, l_r, l_b, d_r, d_b`) and labels in the form of location names (`r_r, r_b`) that will be used to check formal properties in Section 6.2. Finally, the `Receiver` automaton is in charge of resetting the clock when it reaches the `TIMELIMIT` plus the maximum `DEADLINE` allowed for the packets (right-top part). This operation begins with the application consuming the buffer and saving relevant information for the formal validation. When this operation concludes, the clock is reset, along with other complementary variables. All automata keep working following the Wolper test.

Listing 1 contains the values of the constants used in the model, variables for the formal verification (denoted by underscores) and the initial values of the configurable variables (starting with `c_`) that will be used later to study the performance properties with UPPAALSMC. The different functions shown in the model have self-explanatory names; however, the full code of the model can be found in [45].

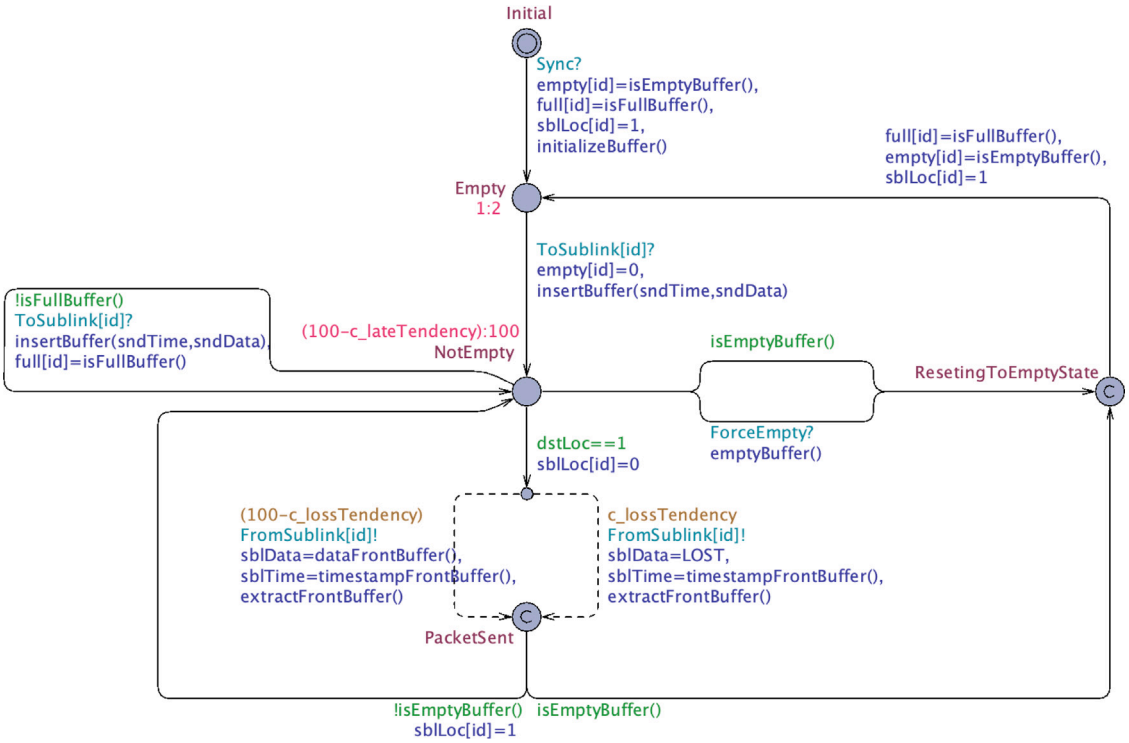


Fig. 12. Sublink automaton.

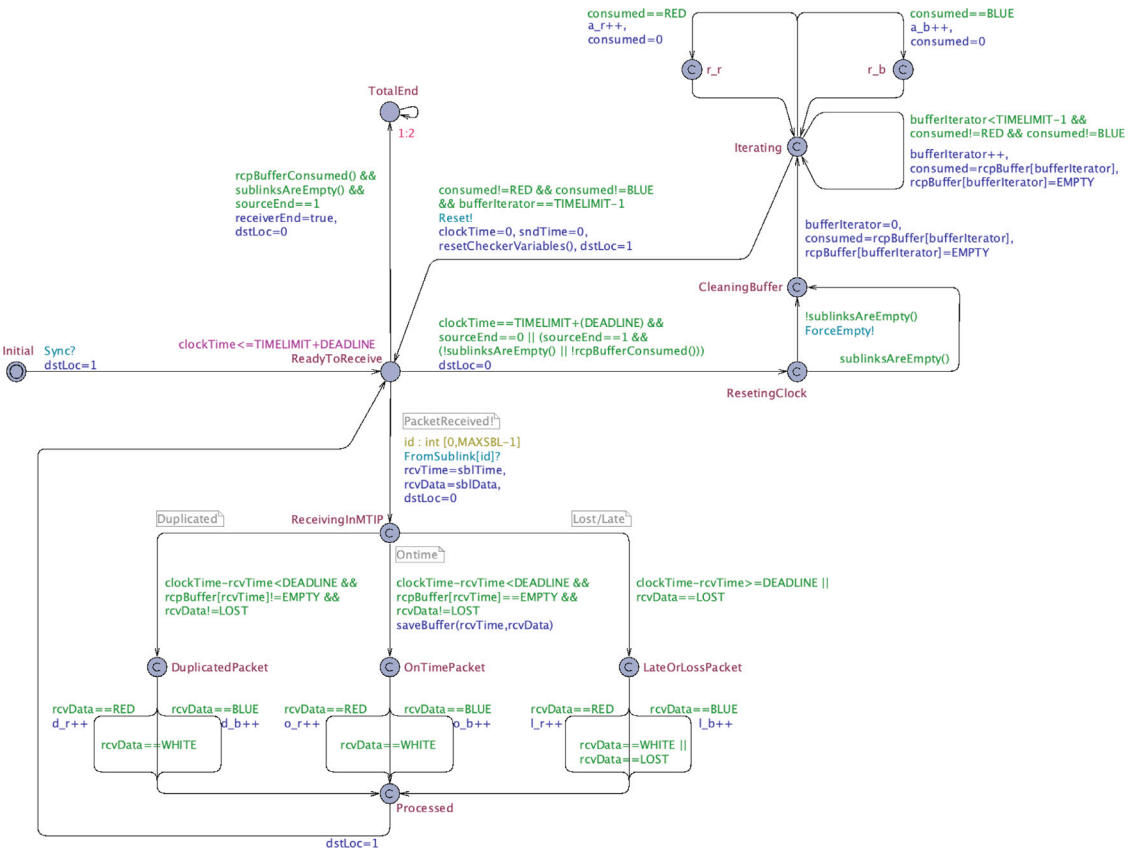


Fig. 13. Receiver automaton.

```
// Constants (Wolper)
const int LOST = 0;
const int WHITE = 1;
const int RED = 2;
const int BLUE = 3;

// Constants (and limits)
const int TIMELIMIT  = 4;
const int MAXBUF = TIMELIMIT;
const int MAXSBL = 3;
const int DEADLINE = 3;
const int EMPTY = 0;

// Variables for formal verification
int a_r, a_b;
int o_r, o_b;
int l_r, l_b;
int d_r, d_b;

// Configurations for SMC
int c_sblNum=3;
int c_lossTendency=50;
int c_lateTendency=50;
```

Listing 1: Global declarations

6.2. Simulation and formal verification of MTIP properties

We run the automata network with UPPAAL. First, we simulate the model to identify and eliminate any discrepancies that could render it an unsuitable representation of MTIP. Then, we evaluate the correctness and performance properties described in Section 4 to check that the UPPAAL model built behaves as expected wrt the MTIP protocol and evaluate its performance.

6.2.1. Simulation of the model

We have generated multiple Message Sequence Charts (MSC) that provide illustrative traces of iterations of our model. These MSCs are represented by UPPAAL as shown in the MSC example of Fig. 14. They visually demonstrate the interconnected interactions between the sender and the receiver, utilizing three distinct sublinks.

In the MSC presented in Fig. 14, we observe the sender transmitting white data through the three separate sublinks. Subsequently, sublink 1 forwards this data to the receiver, which processes it. Afterward, sublink 0 redundantly forwards the same data, resulting in its rejection as a duplicate packet upon reception by the receiver. Finally, sublink 3 forwards the data, which, in this case, is either delayed or lost and serves no purpose for the receiver. The final part of the excerpt of the MSC displayed illustrates the selection of red data, continuing the Wolper test.

It is important to highlight that all MSC analyzed exhibit no deviations from the expected behavior of MTIP, previously depicted in Fig. 7. Consequently, it serves as a foundational reference point for validating our model's adherence to the protocol.

6.2.2. Analysis of correctness properties

We translate the correctness properties introduced in Section 4.1 into the TCTL language used by the UPPAAL validator. The formulas are presented in Listing 2, with `r` being the `Receiver`. In order to check that the application never receives messages that have met their deadline, the formula for **P1** states that if all red or blue packets arrive late, the application will not receive them. The formula for **P2** checks the absence of duplicates through no red or blue data arriving twice to the applications. Then, the formula for **P3** assesses that no red data is received in the application if blue data has been received, to ensure order. Finally, the formula for **P4** checks that if red or blue data arrives on time and is not a duplicate, it will be processed by the application

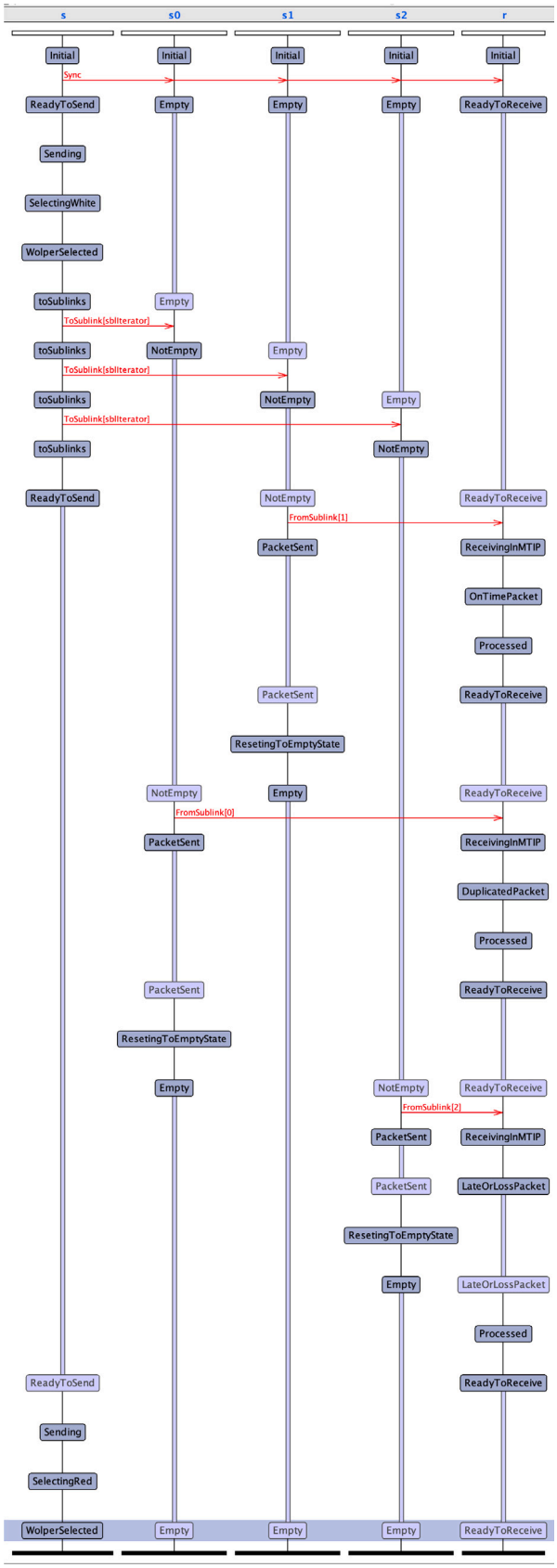


Fig. 14. MSC extracted from UPPAAL.

at some point. We successfully verified the formulas against the models and obtained the results reported in Table 2. The table includes four columns, with the first column "States, stored" indicating the number of unique states discovered during the verification process, reflecting the model's actual size. The second column, labeled "States, matched", lists the number of duplicate or previously encountered states. The sum of these two columns provides a total count of unique transitions performed, which serves as a measure of the verification's workload. The last column shows the memory usage during each verification. It is worth noting that all properties were verified using the same number of stored states, since UPPAAL performs a complete system analysis for each property. However, the matched states were greater for the final properties due to the increased number of states generated when evaluating the "imply" condition.

```
tctl P1  { A[] not ((l_r == c_sblNum && a_r > 0)
    ↪       || (l_b == c_sblNum && a_b > 0)) }
tctl P2  { A[] ((a_r <= 1) && (a_b <= 1)) }
tctl P3  { A[] not (r.r_r && a_b>0) }
tctl P4_red{ (o_r == 1 && a_r == 0) --> r.r_r }
tctl P4_blue{ (o_b == 1 && a_b == 0) --> r.r_b }
```

Listing 2: Correctness properties in TCTL

Table 2
Verification results in UPPAAL.

	States, stored	States, matched	Memory usage
General	42105198	133566430	7581.232 MB
P1	42105198	133566430	7588.732 MB
P2	42105198	133566430	7588.732 MB
P3	42105198	133566430	7588.944 MB
P4_red	42105198	227928967	10735.344 MB
P4_blue	42105198	235998863	10933.564 MB

6.2.3. Analysis of performance properties

After evaluating the correctness of MTIP with UPPAAL, we proceed to study the performance properties. In order to do so, we evaluate on UPPAAL SMC the impact of the modification of the three configurable variables (presented in previous Listing 1), the number of sublinks that are in use, the tendency to losses in the sublinks and the tendency to add additional delays in the sublinks. The idea is to study how using different sublinks under different channel characteristics affects MTIP's two main performance properties, i.e., the probability of receiving packets successfully (**P5**) and the probability of receiving duplicate packets and thus, wasting resources (**P6**). These properties are also based in the Wolper tests and are presented in Listing 3. Since in UPPAAL SMC time is needed for the evaluations, we tested different values and selected the value `time = 100` that assures that the completion of the experiment (`Pr[<=100] (<> receiverEnd == 1)`) occurs in [0.990031,1] with confidence 0.99.

```
tctl P5_red { Pr[<=100] (<> a_r > 0) }
tctl P5_blue{ Pr[<=100] (<> a_b > 0) }
tctl P6_red { Pr[<=100] (<> d_r > 0) }
tctl P6_blue{ Pr[<=100] (<> d_b > 0) }
```

Listing 3: Performance properties in TCTL

First, we evaluate the use of 1, 2 or 3 sublinks with the same conditions and configurations of 0 to 100 loss and late tendencies (in 10-sized steps). The heatmaps that present the first property evaluations are in Fig. 15 while the second property heatmaps are in Fig. 16. These graphs show the mean results of the validation of the properties under the statistical parameter configurations shown in Table 3. To compare both heatmaps graphically, the color blue has been set in both situations to represent a good scenario for that performance property, while the red one represents a bad outcome.

Table 3
SMC parameters in UPPAAL.

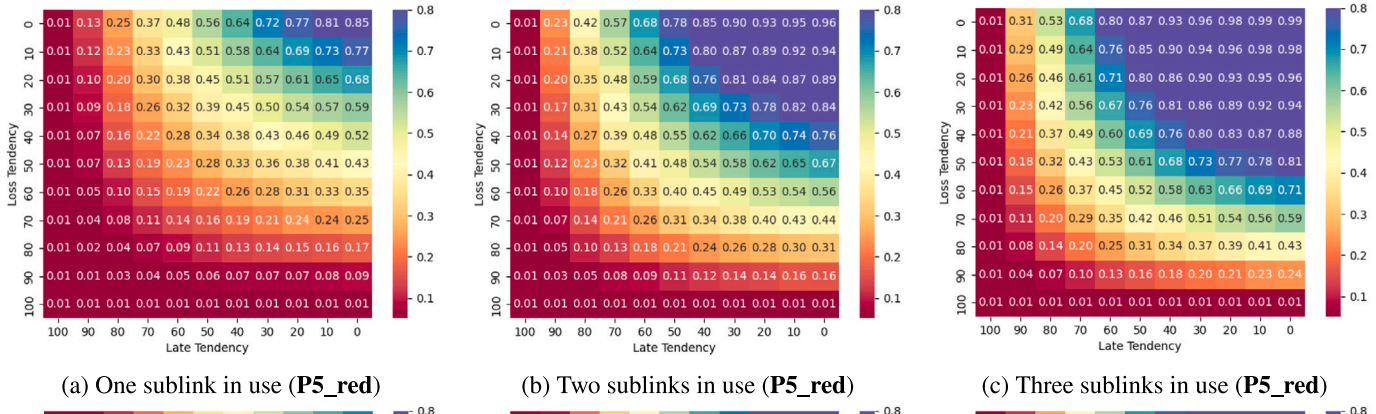
Statistical parameter	Value
Lower probabilistic deviation	0.01
Upper probabilistic deviation	0.01
Probability of false negatives	0.01
Probability of false positives	0.01
Probability uncertainty	0.01
Ratio lower bound	0.9
Ratio upper bound	1.1
Histogram bucket width	0.0
Histogram bucket count	0
Trace resolution	4096
Discretization step for hybrid systems	0.01

Evaluating performance property **P5**, we can extract several ideas. First, a general view of the colors of the graphs shows that the use of more sublinks enhances the communication. More green and blue colors can be seen in scenarios with two and three sublinks. However, it is also clear that when the scenarios are really bad, it is harder to improve them with multiple sublinks, since if all sublinks are faulty, this redundancy cannot overcome the communication issues (see `LossTendency 100%` and `LateTendency 100%`). Moreover, we see that the data of property **P5_red** has slightly more chance of reaching the destination than the data of **P5_blue**. Since the wolper test first sends the RED data and then the BLUE one, it is more likely that just the RED one arrives when evaluation of the property is completed, even if the experiment is completed in [0.990031, 1] probability with confidence 0.99, as stated before.

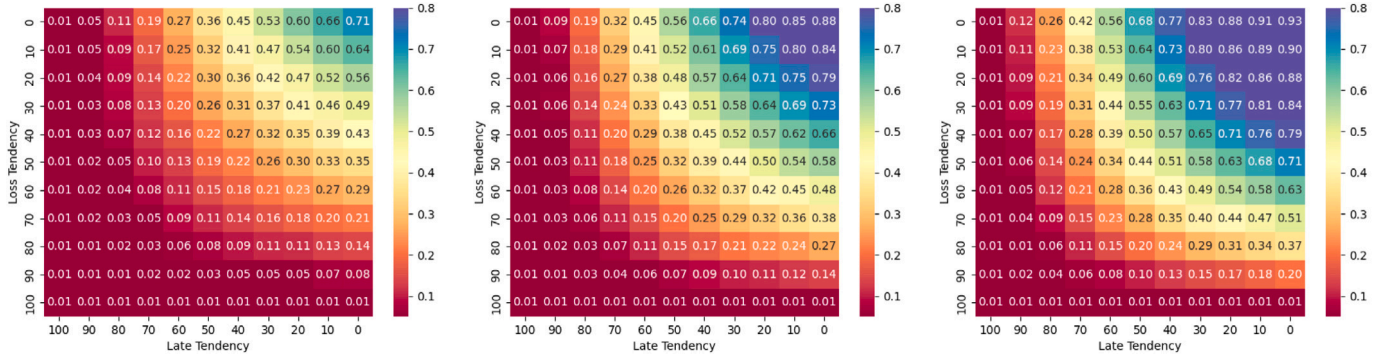
Moving to the performance property **P6**, we observe an obvious analogous behavior. With this property, we want to measure the excess use of resources in the sense of using multiple sublinks. When just one sublink is in use, there are no duplicate packets arriving at the destination, so in this sense there is no excess of resources. However, if two and three sublinks are used, more duplicate packets arrive at the receiving endpoint. Although the duplication of packets helps in the delivery of messages, the fact that duplicate packets are arriving shows that the configuration under study is potentially using more resources than it should.

When looking at both figures as a whole, MTIP's main trade-off is clearly shown. The use of more sublinks helps improve the probability of arrival; however, it also creates more duplicate packets that incur more excessive resource usage. The solution of this trade-off resides in the application's specific requirements and the specific scenario through which it is sending data. For instance, if the requirements set a minimum of 0.9 probability of arrival of red data under a scenario of 20% loss and 10% late tendencies, the test shown in Fig. 15 displays that this can be achieved with three sublinks. However, if the required probability of arrival it is a little looser, such as a 0.85, the maximum resource waste desired also fits in the equation. For example, in the case of a maximum probability of 0.5 duplicates (wrt Fig. 16), the use of two sublinks would be more beneficial for the scenario than the use of three sublinks, even if the latter provides more reliability.

Then, we conducted a second evaluation, exploring the scenario of dissimilar conditions within different sublinks. In this case, the model was adapted to configure distinct values of c_lateTendency and c_lossTendency for each sublink. Given the substantial number of scenarios that emerge from combining 10-sized step configurations across dissimilar sublinks, we chose to focus on three representative sublinks that incorporate loss and late tendencies at levels of 20%, 50%, and 80%. These resulting scenarios simulate a simplified version of MTIP's sending algorithm as presented in [4], selecting the best sublinks based on both loss and late tendencies incrementally. The results are illustrated in Figs. 17 and 18. The nomenclature "X–Y–Z" on the axis

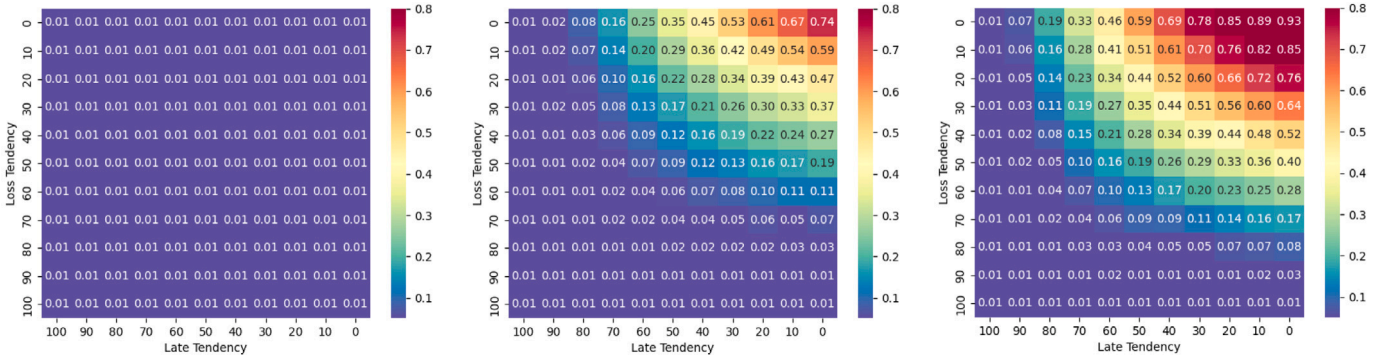


(a) One sublink in use (**P5_red**) (b) Two sublinks in use (**P5_red**) (c) Three sublinks in use (**P5_red**)

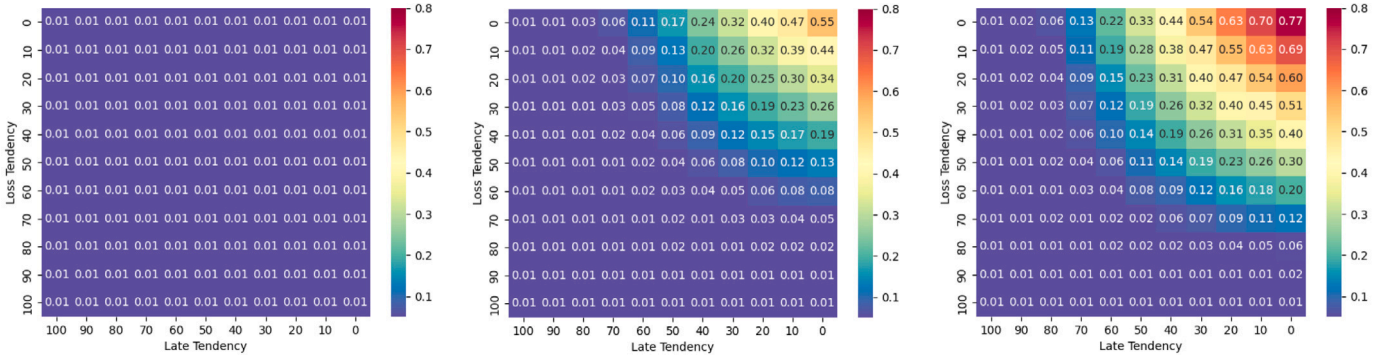


(d) One sublink in use (**P5_blue**) (e) Two sublinks in use (**P5_blue**) (f) Three sublinks in use (**P5_blue**)

Fig. 15. Evaluation of performance property **P5** in sublinks with the same conditions.

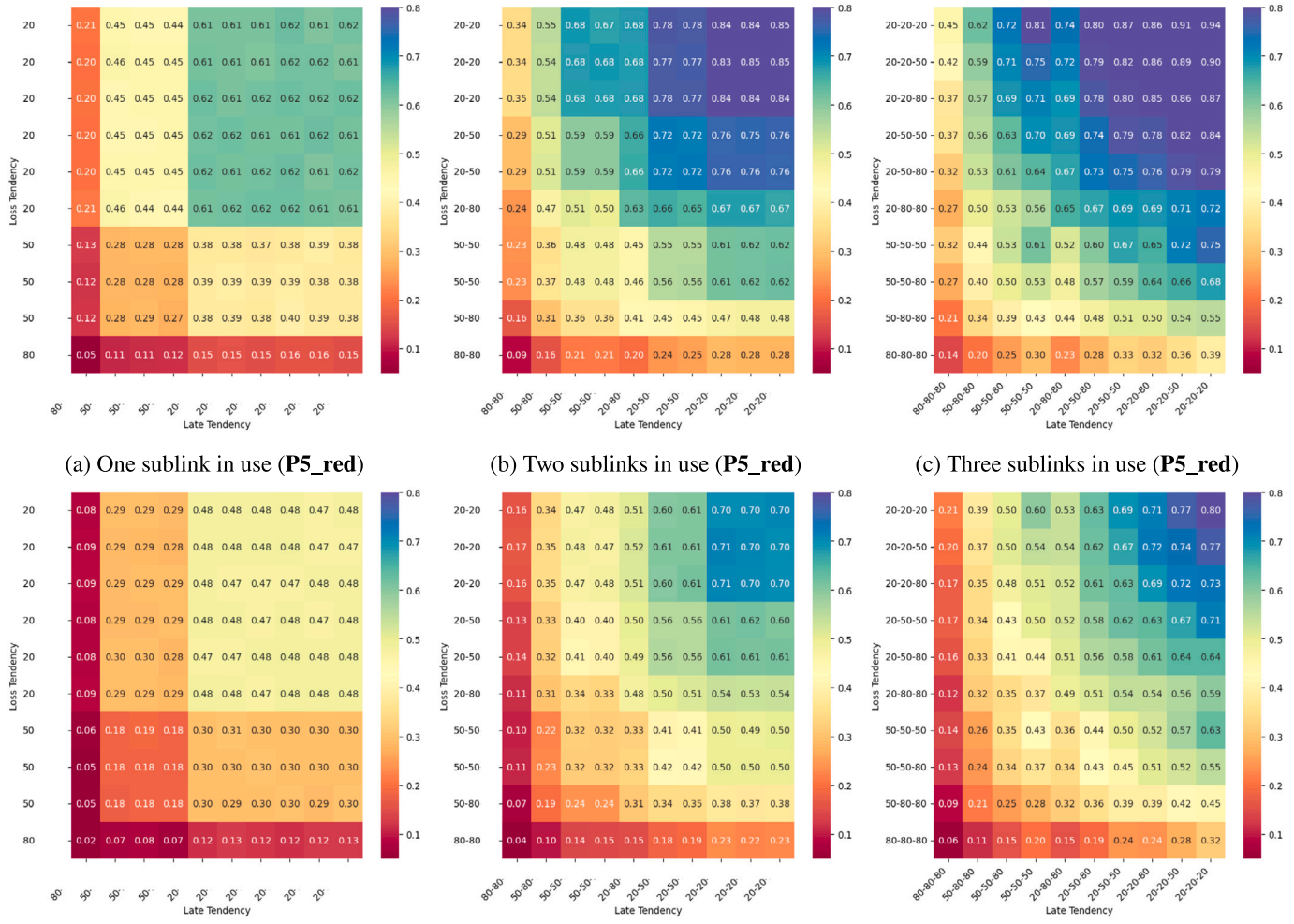


(a) One sublink in use (**P6_red**) (b) Two sublinks in use (**P6_red**) (c) Three sublinks in use (**P6_red**)



(d) One sublink in use (**P6_blue**) (e) Two sublinks in use (**P6_blue**) (f) Three sublinks in use (**P6_blue**)

Fig. 16. Evaluation of performance property **P6** in sublinks with the same conditions.



(a) One sublink in use (**P5_red**) (b) Two sublinks in use (**P5_red**) (c) Three sublinks in use (**P5_red**)

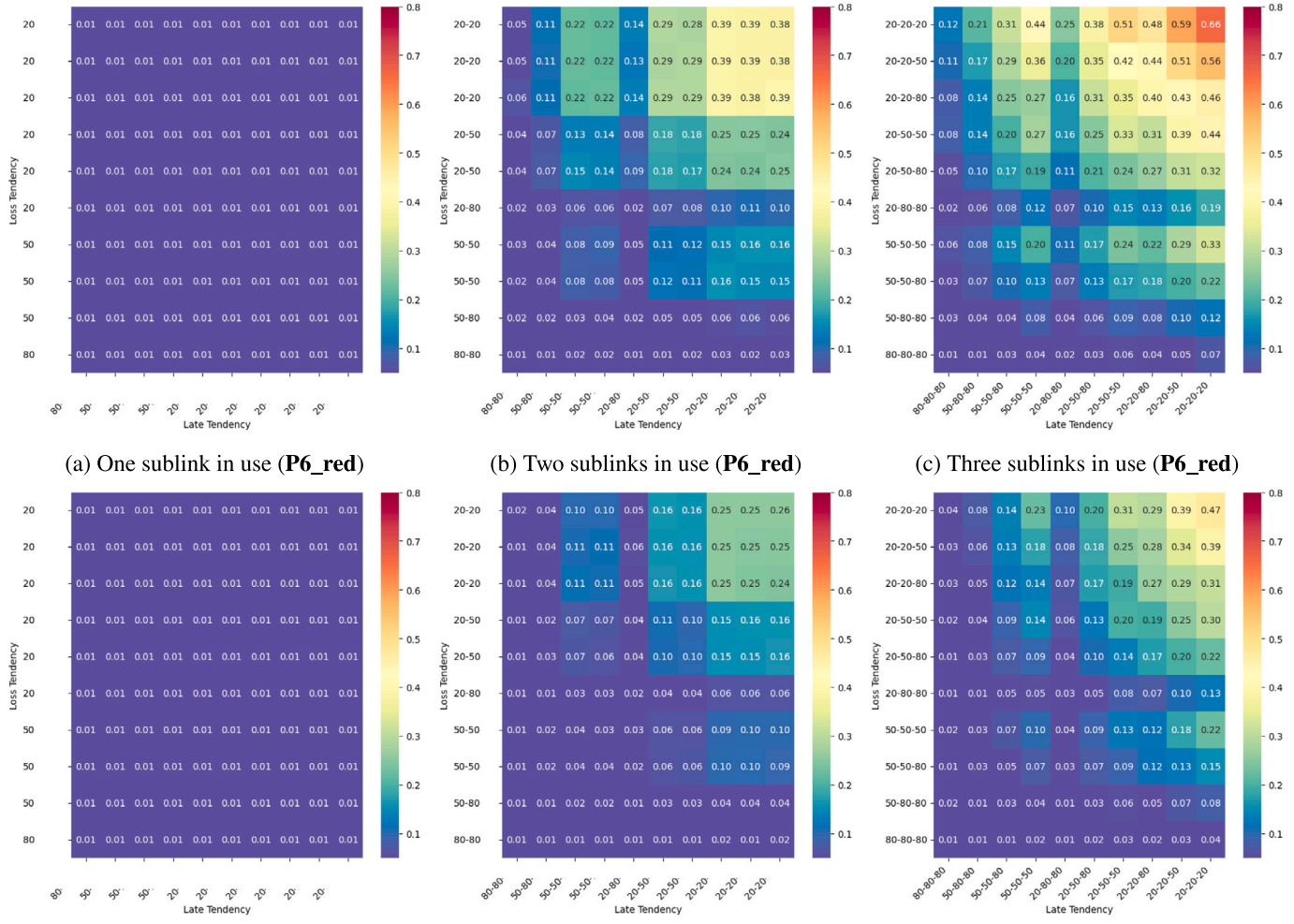
(d) One sublink in use (**P5_blue**) (e) Two sublinks in use (**P5_blue**) (f) Three sublinks in use (**P5_blue**)

Fig. 17. Evaluation of performance property **P5** in sublinks with different conditions.

identifies the loss and late tendencies of the first sublink (X), the second one (Y) and the third one (Z).

From a broader perspective, the conclusions we can draw from the graphs mirror those previously presented: employing more sublinks increases the number of correct messages (**P5**) but also increases the reception of duplicate packets (**P6**). However, this scenario with dissimilar conditions allows for a richer assessment. To illustrate it, we can compare two representative cases in **P5_red** and **P6_red**: case (a) with 20–80–80 loss and late tendencies and case (b) with 50–50–80. In case (a), the first sublink has 20% loss-late tendencies and the second and third sublinks **are** substantially worse compared to it (both 80% loss-late tendencies). In case (b), the first sublink has 50% loss-late tendencies and the second and third sublinks **are not** substantially worse (50% and 80% loss-late tendencies). In case (a), we observe that using more sublinks does not greatly improve the communication. With one sublink, **P5_red** is 0.61, using two sublinks it is 0.63 and using all of them for the communication it is 0.65. In this case, the scenario presents a substantially better sublink, so the addition of further worse sublinks do not greatly improve performance. However, in case (b), we observe that using more sublinks has a noticeable positive impact, with **P5_red** increasing from 0.28 to 0.50 (one sublink vs three sublinks). In fact, the most significant performance improvement occurs when employing two sublinks (increasing from 0.28 to 0.48). This underscores the importance of introducing sublinks with favorable conditions as a key factor in enhancing performance, since the second sublink presents 50% loss-late tendencies while the third sublink shows a level of 80%. Regarding **P6_red**, we observe that the increase of duplicates is highly tied to the increase in performance. In case (a) it goes from 0.01 to 0.02 and then 0.07; whereas in case (b) it rises from 0.01 to 0.08 and finally 0.10. The selection of the specific use of sublinks would depend once again on the application preferences regarding resource waste.

This evaluation serves as a valuable extension of the experiments detailed in [4], in which an initial implementation of MTIP underwent evaluation in a real 5G testbed. In said study, only a limited number of scenarios could be explored due to the constraints of real-world experimentation. Nevertheless, the tests performed showed the main trade-off of MTIP, demonstrating that utilization of additional duplicate packets helps to reduce losses in most scenarios. The performance evaluation of MTIP's UPPAAL model presented in this paper further complements these findings with a comprehensive analysis across a broader spectrum of scenarios. Moreover, it offers valuable insights for potential adjustments to fine-tune MTIP's algorithm, aiding in the selection of the most suitable sublinks for any given scenario, thus improving the performance.



(a) One sublink in use (**P6_red**) (b) Two sublinks in use (**P6_red**) (c) Three sublinks in use (**P6_red**)

(d) One sublink in use (**P6_blue**) (e) Two sublinks in use (**P6_blue**) (f) Three sublinks in use (**P6_blue**)

Fig. 18. Evaluation of performance property **P6** in sublinks with different conditions.

7. Conclusions and future work

MTIP uses multi-connectivity in multi-homed Tactile Internet devices connected to different wireless networks to increase reliability and reduce latency through the selection of the most reliable and fastest paths. The real-time nature of data in this application domain suggests avoiding flow control mechanisms in favor of low latency. In this paper, we have presented a formal model with abstraction decisions to accurately model the protocol and the verification work that ensure correctness and performance. Comparing this work with our previous SPIN based research in [3], we have found that UPPAAL modeling of time is more natural for this protocol and additionally checks the statistical performance properties of the protocol under different configurations.

With the performance evaluation, we have observed the specific impact of using a greater number of sublinks in different scenarios. In most cases, this leads to an improved reliability at the expense of some resource waste. This work complements the evaluation of MTIP's implementation in [4] and gives some insight for future adjustments to fine-tune MTIP's algorithm to select the most appropriate sublinks. Moreover, the UPPAAL model presented here opens up future work to exploit statistical model checking for MTIP in different networking scenarios. Considering the growing interest in Non-Terrestrial Networks (NTN) in the context of cellular networks [48,49], a relevant example is to evaluate the behavior of MTIP combining 5G with satellite links without the elevated cost of a satellite connection.

CRediT authorship contribution statement

Delia Rico: Conceptualization, Methodology, Software, Validation, Formal analysis, Writing – original draft. **María-del-Mar Gallardo:** Conceptualization, Methodology, Validation, Formal analysis, Writing – review & editing. **Pedro Merino:** Conceptualization, Methodology, Validation, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

Work supported by the 6G-SANDBOX Project (Smart Networks and Services Joint Undertaking under the European Union's Horizon 2020 research and innovation programme grant agreement No 101096328), and by the Spanish government (grant FPU17/04292 and the UNICO I+D 5G+TACTILE_1 project TSI-063000-2021-11). Funding for open access charge: Universidad de Málaga / CBUA.

References

[1] O. Holland, E. Steinbach, R.V. Prasad, Q. Liu, Z. Dawy, A. Aijaz, N. Pappas, K. Chandra, V.S. Rao, S. Oteafy, M. Eid, M. Luden, A. Bhardwaj, X. Liu, J. Sachs, J. Araújo, The IEEE 1918.1 "tactile internet" standards working group and its standards, Proc. IEEE 107 (2) (2019) 256–279, http://dx.doi.org/10.1109/JPROC.2018.2885541.

[2] V. Singh, T. Karkkainen, J. Ott, S. Ahsan, L. Eggert, Multipath RTP (MPRTP), Tech. Rep., (draft-singh-avtcore-mprtp) IETF Secretariat, 2017, URL https://datatracker.ietf.org/doc/html/draft-ietf-avtcore-mprtp.

[3] D. Rico, M.-d.-M. Gallardo, P. Merino, Modeling and verification of the multi-connection tactile internet protocol, in: Proceedings of the 17th ACM Symposium on QoS and Security for Wireless and Mobile Networks, Q2SWinet '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 105–114, http://dx.doi.org/10.1145/3479242.3487328.

[4] D. Rico, K.-J. Grinemmo, A. Brunstrom, P. Merino, Performance analysis of the multi-connection tactile internet protocol over 5G, J. Netw. Syst. Manage. 31 (3) (2023) 49, http://dx.doi.org/10.1007/s10922-023-09737-0.

[5] 3GPP, 3rd Generation Partnership Project; Technical Specification Group Core Network and Terminals; General Packet Radio System (GPRS) Tunnelling Protocol User Plane (GTPV1-U) (Release 17), Tech. Rep., (29.281) 3rd Generation Partnership Project, 2021, Version 17.0.0.

[6] V. Shankarkumar, L. Montini, T. Frost, G. Dowd, Precision Time Protocol Version 2 (PTPV2) Management Information Base, Tech. Rep., (8173) RFC Editor, 2017.

[7] G.J. Holzmann, Design and Validation of Computer Protocols, vol. 512, Prentice hall Englewood Cliffs, 1991.

[8] K. Hameed, S. Garg, M.B. Amin, B. Kang, Towards a formal modelling, analysis and verification of a clone node attack detection scheme in the internet of things, Comput. Netw. 204 (2022) 108702, http://dx.doi.org/10.1016/j.comnet.2021.108702.

[9] G.J. Holzmann, The model checker SPIN, IEEE Trans. Softw. Eng. 23 (5) (1997) 279–295, http://dx.doi.org/10.1109/32.588521.

[10] P. Wolper, Expressing interesting properties of programs in propositional temporal logic, in: Proceedings of the 13th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, in POPL '86: Proceedings of the 13th ACM SIGACT-SIGPLAN symposium on Principles of programming languages, New York, NY, USA, 1986, pp. 184–193, http://dx.doi.org/10.1145/512644.512661.

[11] K.S. Kim, D.K. Kim, C. Chae, S. Choi, Y. Ko, J. Kim, Y. Lim, M. Yang, S. Kim, B. Lim, K. Lee, K.L. Ryu, Ultrareliable and low-latency communication techniques for tactile internet services, Proc. IEEE 107 (2) (2019) 376–393, http://dx.doi.org/10.1109/JPROC.2018.2868995.

[12] 3GPP, 3rd Generation Partnership Project; Technical Specification Group Services and System Aspects; Service Requirements for Cyber-Physical Control Applications in Vertical Domains; Stage 1 (Release 18), Tech. Rep., (22.104) 3rd Generation Partnership Project, 2021, Version 18.0.0.

[13] 3GPP, 3rd Generation Partnership Project; Technical Specification Group Services and System Aspects; Service Requirements for the 5G System; Stage 1 (Release 16), Tech. Rep., (22.261) 3rd Generation Partnership Project, 2017, Version 16.0.0.

[14] G. Cisotto, E. Casarin, S. Tomasin, Requirements and enablers of advanced healthcare services over future cellular systems, IEEE Commun. Mag. 58 (3) (2020) 76–81, http://dx.doi.org/10.1109/MCOM.001.1900349.

[15] P.M. Vespa, C. Miller, X. Hu, V. Nenov, F. Buxey, N.A. Martin, Intensive care unit robotic telepresence facilitates rapid physician response to unstable patients and decreased cost in neurointensive care, Surg. Neurol. 67 (4) (2007) 331–337, http://dx.doi.org/10.1016/j.surneu.2006.12.042.

[16] S. Dadhich, U. Bodin, U. Andersson, Key challenges in automation of earth-moving machines, Autom. Constr. 68 (2016) 212–222, http://dx.doi.org/10.1016/j.autcon.2016.05.009.

[17] S. Dodeller, Transport layer protocols for telehaptics update messages, in: Proc. 22nd Biennial Symposium on Communications, June 2004, 2004.

[18] A. Cen, M.W. Mutka, D. Zhu, N. Xi, Supermedia transport for teleoperations over overlay networks, in: NETWORKING 2005: Networking Technologies, Services, and Protocols, in: Lecture Notes in Computer Science, vol. 3462, Springer, 2005, pp. 1409–1412, http://dx.doi.org/10.1007/11422778_126.

[19] R. Wirz, M. Ferre, R. Marín, J. Barrio, J.M. Claver, J. Ortego, Efficient transport protocol for networked haptics applications, in: M. Ferre (Ed.), Haptics: Perception, Devices and Scenarios, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 3–12.

[20] G. Kokkonis, K. Psannis, M. Roumeliotis, S. Kontogiannis, A survey of transport protocols for haptic applications, in: Proceedings of the 16th Panhellenic Conference on Informatics, 2012, pp. 192–197, http://dx.doi.org/10.1109/PCi.2012.54.

[21] J. Qadir, A. Ali, K.A. Yau, A. Sathiaseelan, J. Crowcroft, Exploiting the power of multiplicity: A holistic survey of network-layer multipath, IEEE Commun. Surv. Tutor. 17 (4) (2015) 2176–2213, http://dx.doi.org/10.1109/COMST.2015.2453941.

[22] D. Rico, P. Merino, A survey of end-to-end solutions for reliable low-latency communications in 5G networks, IEEE Access 8 (2020) 192808–192834, http://dx.doi.org/10.1109/ACCESS.2020.3032726.

[23] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, TCP Extensions for Multipath Operation with Multiple Addresses, Tech. Rep., (8684) RFC Editor, 2020, http://www.rfc-editor.org/rfc/rfc8684.txt.

[24] T. Viernickel, A. Froemmgen, A. Rizk, B. Koldehofe, R. Steinmetz, Multipath QUIC: A deployable multipath transport protocol, in: 2018 IEEE International Conference on Communications, 2018, pp. 1–7, http://dx.doi.org/10.1109/ICC.2018.8422951.

[25] C. Battaglia, V. Bruyère, O. Gauwin, C. Pelsser, B. Quoitin, Reasoning on BGP routing filters using tree automata, Comput. Netw. 65 (2014) 232–254, http://dx.doi.org/10.1016/j.comnet.2014.01.001.

[26] B. Touijer, Y. Ben Maissa, S. Mouline, IEEE 802.15.6 CSMA/CA access method for WBANs: Performance evaluation and new backoff counter selection procedure, Comput. Netw. 188 (2021) 107759, http://dx.doi.org/10.1016/j.comnet.2020.107759.

[27] E.K.K. Edris, M. Aiash, J.K.-K. Loo, Formal verification and analysis of primary authentication based on 5G-AKA protocol, in: 2020 Seventh International Conference on Software Defined Systems, SDS, 2020, pp. 256–261, http://dx.doi.org/10.1109/SDS49854.2020.9143899.

[28] M. Kamel, S. Leue, Formalization and validation of the general inter-ORB protocol (GIOP) using PROMELA and SPIN, Int. J. Softw. Tools Technol. Transf. 2 (4) (2000) 394–409.

[29] P. Merino, J.M. Troya, Modeling and verification of the ITU-T multipoint communication service with SPIN, in: Proceedings of the 2nd International Wrokshop on SPIN Verification, 1996.

[30] Y. Cai, D. Qi, Control protocols design for cyber-physical systems, in: 2015 IEEE Advanced Information Technology, Electronic and Automation Control Conference, IAEAC, 2015, pp. 668–671, http://dx.doi.org/10.1109/IAEAC.2015.7428638.

[31] S. Chouali, A. Boukerche, A. Mostefaoui, M.A. Merzoug, Formal verification and performance analysis of a new data exchange protocol for connected vehicles, IEEE Trans. Veh. Technol. 69 (12) (2020) 15385–15397, http://dx.doi.org/10.1109/TVT.2020.3040817.

[32] UP4ALL International AB, Uppaal, 2022, https://uppaal.org/. (Accessed 18 November 2022).

[33] S. Saini, A. Fehnker, Evaluating the stream control transmission protocol using uppaal, Electron. Proc. Theor. Comput. Sci. 244 (2017) 1–13, http://dx.doi.org/10.4204/eptcs.244.1.

[34] A. Rashid, O. Hasan, K. Saghar, Formal analysis of a ZigBee-based routing protocol for smart grids using UPPAAL, in: 2015 12th International Conference on High-Capacity Optical Networks and Enabling/Emerging Technologies, HONET, 2015, pp. 1–5, http://dx.doi.org/10.1109/HONET.2015.7395420.

[35] X. Wu, H. Ling, Y. Dong, On modeling and verifying of application protocols of TTCAN in flight-control system with UPPAAL, in: 2009 International Conference on Embedded Software and Systems, 2009, pp. 572–577, http://dx.doi.org/10.1109/ICESS.2009.27.

[36] W. Guo, Y. Huang, J. Shi, Z. Hou, Y. Yang, A formal method for evaluating the performance of TSN traffic shapers using UPPAAL, in: 2021 IEEE 46th Conference on Local Computer Networks, LCN, 2021, pp. 241–248, http://dx.doi.org/10.1109/LCN52139.2021.9524955.

[37] M. Houimli, L. Kahloul, S. Benaoun, Formal specification, verification and evaluation of the MQTT protocol in the Internet of Things, in: 2017 International Conference on Mathematics and Information Technology, ICMIT, IEEE, 2017, pp. 214–221.

[38] M. Houimli, L. Kahloul, S. Benaoun, Formal specification, verification and evaluation of the MQTT protocol in the Internet of Things, in: 2017 International Conference on Mathematics and Information Technology, ICMIT, 2017, pp. 214–221, http://dx.doi.org/10.1109/MATHIT.2017.8259720.

[39] P. Höfner, A. McIver, Statistical model checking of wireless mesh routing protocols, in: NASA Formal Methods: 5th International Symposium, NFM 2013, Moffett Field, CA, USA, May 14-16, 2013. Proceedings 5, Springer, 2013, pp. 322–336.

[40] M. Naeem2023, M. Albano, K.G. Larsen, B. Nielsen, A. HOedholt, C.O. Laursen, Modelling and analysis of a sigfox based IoT network using UPPAAL SMC, IEEE Sens. J. (2023) 1, http://dx.doi.org/10.1109/JSEN.2023.3261667.

[41] G. Behrmann, A. David, K.G. Larsen, A tutorial on uppaal, in: M. Bernardo, F. Corradini (Eds.), Formal Methods for the Design of Real-Time Systems: International School on Formal Methods for the Design of Computer, Communication, and Software Systems, Bertinora, Italy, September 13-18, 2004, Revised Lectures, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 200–236, http://dx.doi.org/10.1007/978-3-540-30080-9_7.

[42] O. Holland, E. Steinbach, R.V. Prasad, Q. Liu, Z. Dawy, A. Aijaz, N. Pappas, K. Chandra, V.S. Rao, S. Oteafy, M. Eid, M. Luden, A. Bhardwaj, X. Liu, J. Sachs, J. Araújo, The IEEE 1918.1 "tactile internet" standards working group and its standards, Proc. IEEE 107 (2) (2019) 256–279, http://dx.doi.org/10.1109/JPROC.2018.2885541.

[43] M. Amend, D. Hugo, Multipath Sequence Maintenance, Tech. Rep., (draft-amend-iccrg-multipath-reordering-03) IETF Secretariat, 2022, https://www.ietf.org/archive/id/draft-amend-iccrg-multipath-reordering-03.txt.

[44] G.J. Holzmann, The SPIN Model Checker - Primer and Reference Manual, Addison-Wesley, 2004, pp. I–XII, 1–596.

[45] D. Rico, MTIP: The multi-connection tactile internet protocol, 2021, URL https://github.com/deliarico/MTIP.
[46] G. Behrmann, A. David, K.G. Larsen, A tutorial on Uppaal 4.0, 2006, Department of computer science, Aalborg university.
[47] A. David, K.G. Larsen, A. Legay, M. Mikučionis, D.B. Poulsen, Uppaal SMC tutorial, Int. J. Softw. Tools Technol. Transf. 17 (4) (2015) 397–415, http://dx.doi.org/10.1007/s10009-014-0361-y.
[48] M. Giordani, M. Zorzi, Non-terrestrial networks in the 6G era: Challenges and opportunities, IEEE Netw. 35 (2) (2020) 244–251.
[49] G. Araniti, A. Iera, S. Pizzi, F. Rinaldi, Toward 6G non-terrestrial networks, IEEE Netw. 36 (1) (2021) 113–120.